Manipulation with ROS2

Unit 3: Doing Motion Planning Programmatically

- Summary -

Estimated time for completion: **4h**

This unit will show you how to perform motion planning using the **Move Group C++ Interface**. You will learn to create a C++ program that communicates with MoveIt2 functionalities in order to perform motion planning.

- End of Summary -

In the previous unit, you saw that you could plan and execute trajectories for your robot using the MoveIt2 RViz environment. But, this is not the typical case.

Usually, you will want to move your robot with your own programs. And this is exactly what you are going to do in this unit! However, you will use C++ to control the robot in this course. The main reason is that when writing this tutorial, MoveIt2 doesn't have a Python API available.

3.1 Planning a trajectory

In motion planning, we can differentiate between two main basic methods:

1. **Planning to a joint-space goal**: In this method, we set the desired joint positions for the arm. The motion planning algorithm computes a trajectory for moving from the current joint positions to the new desired ones. This motion planning method is simpler as it doesn't involve inverse kinematics (IK).
1. **Planning to an end-effector pose**: In this approach, we set the desired pose, which includes both position and orientation, for the end-effector link in space. This motion planning method is more complex as it does involve inverse kinematics (IK).

3.1.1 Planning to a joint-space goal

As you've seen in the previous unit, **there is a difference between planning and executing a trajectory**. So, in this first part of the unit, you will see how to plan a trajectory with C++. In particular, you will see how to plan a joint-space goal. For this, complete the next exercise.

- Exercise 3.1 -

Let's have a quick look at the code that you will be executing:

test_trajectory.cpp

In [ ]:

```
#include <moveit/move_group_interface/move_group_interface.h>
#include <moveit/planning_scene_interface/planning_scene_interface.h>
```

```
#include <moveit_msgs/msg/display_robot_state.hpp>
#include <moveit_msgs/msg/display_trajectory.hpp>

static const rclcpp::Logger LOGGER = rclcpp::get_logger("move_group_demo");
static const std::string PLANNING_GROUP = "ur_manipulator";

int main(int argc, char **argv) {
  rclcpp::init(argc, argv);
  rclcpp::NodeOptions node_options;
  node_options.automatically_declare_parameters_from_overrides(true);
  auto move_group_node =
      rclcpp::Node::make_shared("move_group_interface_tutorial", node_options);

  rclcpp::executors::SingleThreadedExecutor executor;
  executor.add_node(move_group_node);
  std::thread([&executor]() { executor.spin(); }).detach();

  moveit::planning_interface::MoveGroupInterface move_group(move_group_node,
                                                            PLANNING_GROUP);

  const moveit::core::JointModelGroup *joint_model_group =
      move_group.getCurrentState()->getJointModelGroup(PLANNING_GROUP);

  RCLCPP_INFO(LOGGER, "Planning frame: %s",
              move_group.getPlanningFrame().c_str());

  RCLCPP_INFO(LOGGER, "End effector link: %s",
              move_group.getEndEffectorLink().c_str());

  RCLCPP_INFO(LOGGER, "Available Planning Groups:");
  std::copy(move_group.getJointModelGroupNames().begin(),
            move_group.getJointModelGroupNames().end(),
            std::ostream_iterator<std::string>(std::cout, ", "));

  moveit::core::RobotStatePtr current_state = move_group.getCurrentState(10);

  std::vector<double> joint_group_positions;
  current_state->copyJointGroupPositions(joint_model_group,
                                         joint_group_positions);

  // joint_group_positions[0] = 0.00;  // Shoulder Pan
  joint_group_positions[1] = -2.50; // Shoulder Lift
  joint_group_positions[2] = 1.50;  // Elbow
  joint_group_positions[3] = -1.50; // Wrist 1
  joint_group_positions[4] = -1.55; // Wrist 2
  // joint_group_positions[5] = 0.00;  // Wrist 3
```

```
  move_group.setJointValueTarget(joint_group_positions);

  moveit::planning_interface::MoveGroupInterface::Plan my_plan;

  bool success =
      (move_group.plan(my_plan) == moveit::core::MoveItErrorCode::SUCCESS);

  rclcpp::shutdown();
  return 0;
}
```

That's great! But, how does this code work? What does each part mean? Let's break it down into smaller pieces.
First of all we define a couple of static variables for the **logger** system and for the **planning group of our arm**, which is named `ur_manipulator`:

In [ ]:

```
static const rclcpp::Logger LOGGER = rclcpp::get_logger("move_group_demo");
static const std::string PLANNING_GROUP_ARM = "ur_manipulator";
```

Here we are creating a ROS2 node and adding this node to a `SingleThreadedExecutor` executor. Then, we spin this executor until somebody terminates the program.

In [ ]:

```
auto move_group_node =
    rclcpp::Node::make_shared("move_group_interface_tutorial", node_options);

rclcpp::executors::SingleThreadedExecutor executor;
executor.add_node(move_group_node);
std::thread([&executor]() { executor.spin(); }).detach();
```

Next we create the `MoveGroupInterface`:

In [ ]:

```
moveit::planning_interface::MoveGroupInterface move_group(move_group_node,
                                                          PLANNING_GROUP);

moveit::planning_interface::PlanningSceneInterface planning_scene_interface;

const moveit::core::JointModelGroup *joint_model_group =
  move_group.getCurrentState()->getJointModelGroup(PLANNING_GROUP);
```

As you can see, we initialize a `MoveGroupInterface` named `move_group` for the arm planning group. It can be easily setup using just the name of the planning group you would like to control and plan for. This interface will allow you to call the move group node capabilities from a C++ program.
Next we create a `JointModelGroup` pointer:

In [ ]:

```
const moveit::core::JointModelGroup *joint_model_group =
    move_group.getCurrentState()->getJointModelGroup(PLANNING_GROUP);
```

Pointers are often used to refer to the planning group for improved performance. This `JointModelGroup` pointer will contain all the information related to the planning group.
Next we are getting and printing some information related to our setup:

In [ ]:

```
// We print the current planning frame for this group.
RCLCPP_INFO(LOGGER, "Planning frame: %s",
            move_group.getPlanningFrame().c_str());

// We can also print the name of the end-effector link for this group.
RCLCPP_INFO(LOGGER, "End effector link: %s",
        move_group.getEndEffectorLink().c_str());

// We can get a list of all the groups in the robot:
RCLCPP_INFO(LOGGER, "Available Planning Groups:");
std::copy(move_group.getJointModelGroupNames().begin(),
      move_group.getJointModelGroupNames().end(),
      std::ostream_iterator<std::string>(std::cout, ", "));
```

Here we get some data about our MoveIt2 setup. This is not mandatory for the planning, but it can be very useful to make sure everything is configured as it should. Specifically, you are getting data about:

- The name of the frame in which the robot is planning
- The current end-effector link
- The available Planning Groups

Next we get the **`current state`** of our robot:

In [ ]:

```
moveit::core::RobotStatePtr current_state = move_group.getCurrentState(10);

std::vector<double> joint_group_positions;
current_state->copyJointGroupPositions(joint_model_group,
                                       joint_group_positions);
```

What we are doing here is to get the current state of the robot. Then we copy the values of the variable **`joint_model_group`** (which contains the position value of each joint of the robot) into the vector **`joint_group_positions`**.
Finally we plan a motion by changing the position values of the joints:

In [ ]:

```
//joint_group_positions[0] = 0.00;  // Shoulder Pan
```

```
joint_group_positions[1] = -2.50;  // Shoulder Lift
joint_group_positions[2] = 1.50;  // Elbow
joint_group_positions[3] = -1.50;  // Wrist 1
joint_group_positions[4] = -1.55;  // Wrist 2
//joint_group_positions[5] = 0.00;  // Wrist 3
move_group.setJointValueTarget(joint_group_positions);

moveit::planning_interface::MoveGroupInterface::Plan my_plan;

bool success =
  (move_group.plan(my_plan) == moveit::core::MoveItErrorCode::SUCCESS);
```

Here we are doing a couple of important things:

- First, we set the desired position values of the joints. In this case, we are setting the joint values of the **home** position.
- Second, we set these new values as the new target using the `setJointValueTarget` method.
- Finally, we call the `plan()` method of the move group interface to plan the trajectory.

You can have a deeper look at the functions provided by `MoveGroupInterface` here: http://docs.ros.org/en/noetic/api/moveit_ros_planning_interface/html/classmoveit_1_1planning__interface_1_1MoveGroupInterface.html

Let's create the files and launch them

a) Inside your **ros2_ws**, create a new package named **moveit2_scripts** with some dependencies.

Execute in Shell #1

In [ ]:

```
cd ~/ros2_ws/src
ros2 pkg create moveit2_scripts --dependencies rclcpp rclcpp_action moveit_core
moveit_ros_planning moveit_ros_planning_interface interactive_markers geometric_shapes
control_msgs moveit_msgs
```

Inside this package, create a new file named **test_trajectory.cpp** and copy the code you've just seen above inside this file.

In [ ]:

```
cd ~/ros2_ws/src/moveit2_scripts/src
touch test_trajectory.cpp
```

b) Inside the package, also create a launch file to launch the program, called **test_trajectory.launch.py**.

Execute in Shell #1

In [ ]:

```
cd ~/ros2_ws/src/moveit2_scripts
mkdir launch
```

touch launch/test_trajectory.launch.py

Paste the code below into this launch file:

test_trajectory.launch.py

In [ ]:

```
import os
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.actions import ExecuteProcess
from ament_index_python.packages import get_package_share_directory
from moveit_configs_utils import MoveItConfigsBuilder


def generate_launch_description():
    moveit_config = MoveItConfigsBuilder("name",
package_name="my_moveit_config").to_moveit_configs()

    # MoveItCpp demo executable
    moveit_cpp_node = Node(
        name="test_trajectory",
        package="moveit2_scripts",
        executable="test_trajectory",
        output="screen",
        parameters=[
            moveit_config.robot_description,
            moveit_config.robot_description_semantic,
            moveit_config.robot_description_kinematics,
            {'use_sim_time': True},
        ],
    )

    return LaunchDescription(
        [moveit_cpp_node]
    )
```

The launch file is pretty simple. First, you load your MoveIt2 package configuration into the variable `moveit_config`:

In [ ]:

```
moveit_config = MoveItConfigsBuilder("name",
package_name="my_moveit_config").to_moveit_configs()
```

The, you launch the `test_trajectory` executable, which will execute the `test_trajectory.cpp` program:

In [ ]:

```
moveit_cpp_node = Node(
    name="test_trajectory",
    package="moveit2_scripts",
    executable="test_trajectory",
    output="screen",
    parameters=[
        moveit_config.robot_description,
        moveit_config.robot_description_semantic,
        moveit_config.robot_description_kinematics,
        {'use_sim_time': True},
    ],
)
```

As you can see, you need to pass to your program the `robot description`, `robot semantic` and `robot kinematics` data.

c) Now, update the **CMakeLists.txt** file to compile everything. First, open the file and add the following lines to the code:

Add to CMakeLists.txt

In [ ]:

```
# Generate the executable
add_executable(test_trajectory
    src/test_trajectory.cpp)
target_include_directories(test_trajectory
    PUBLIC include)
ament_target_dependencies(test_trajectory
    ament_cmake
            rclcpp
            rclcpp_action
            moveit_core
            moveit_ros_planning_interface
            interactive_markers
            moveit_ros_planning
            control_msgs)

# Install the executable
install(TARGETS
            test_trajectory
    DESTINATION lib/${PROJECT_NAME}
)

# Install the launch file
install(DIRECTORY
            launch
    DESTINATION share/${PROJECT_NAME}
)
```

You are generating an executable from the file **test_trajectory.cpp** and installing it alongside the contents of the launch folder.

d) Great! Let's now compile our workspace.

Execute in Shell #1

In [ ]:

```
cd ~/ros2_ws
colcon build
source install/setup.bash
```

e) Now test your new program! First, you'll need to launch the MoveIt2 RVIZ environment.

- Reminder -

Remember that before being able to use the move group interface, you need to start the move group node and, optionally, moveit rviz for visualization purposes.

Execute in Shell #1

In [ ]:

```
ros2 launch my_moveit_config move_group.launch.py
```

Execute in Shell #2

In [ ]:

```
ros2 launch my_moveit_config moveit_rviz.launch.py
```

- End of Reminder -

f) Now, run your code:

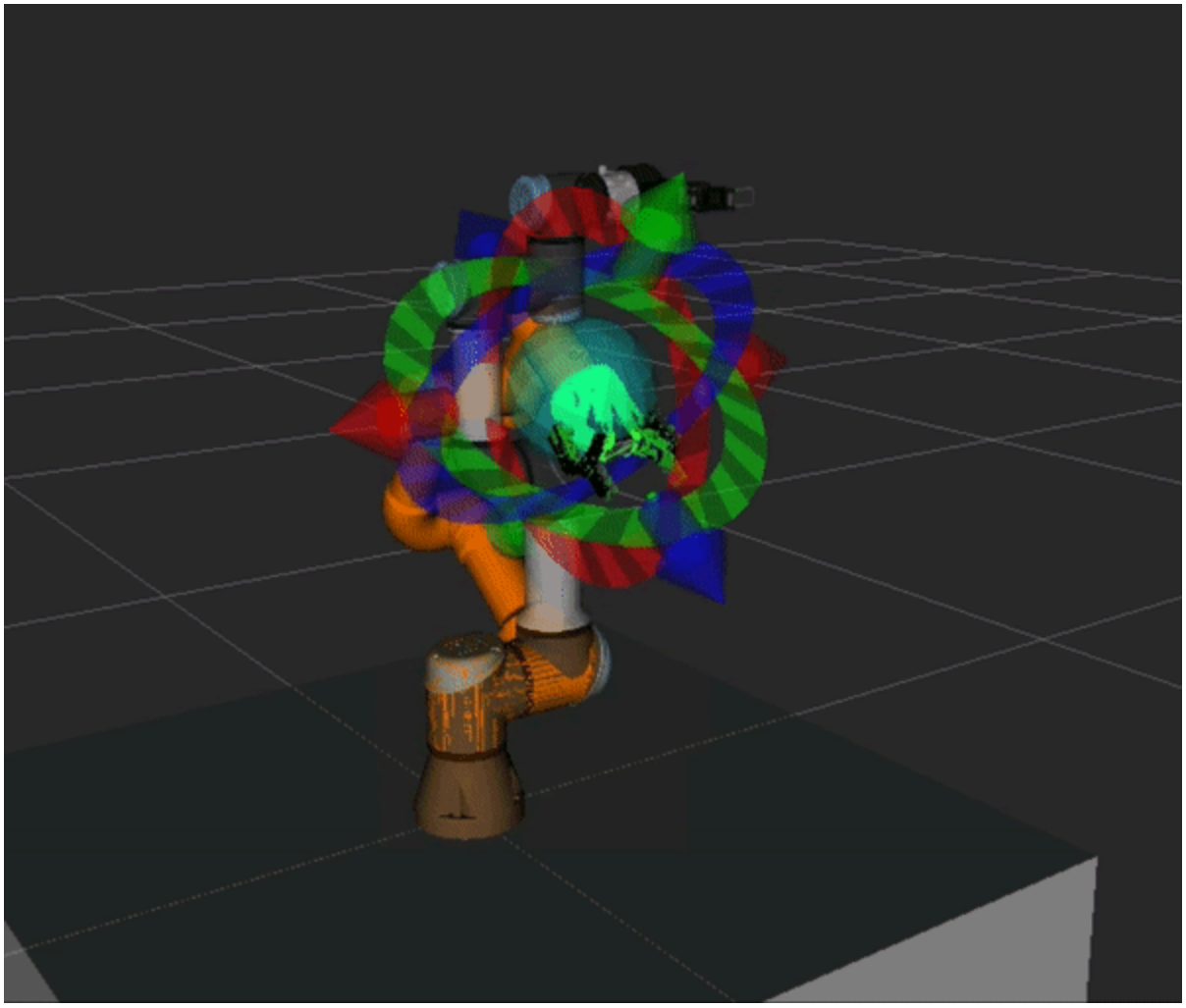
Execute in Shell #3

In [ ]:

```
ros2 launch moveit2_scripts test_trajectory.launch.py
```

You will see how the plan to the **home** position is computed.

- Expected Result for Exercise 3.1 -



- End of Expected Result -

d) You can modify the values assigned to the **joint_group_positions** for planning different motions.

- Note -

REMEMBER: You have just planned a trajectory! You have not executed the trajectory yet. That is why the robot in the simulation is not moving. To get the robot to execute the planned trajectory, you have to make it execute. You'll see how to do that later in this unit.

- End of Note -

- End of Exercise 3.1 -

3.1.2 Planning to an end-effector pose

Sometimes, moving the end-effector towards a desired goal pose might be more convenient. Let's see how you can do this.

- Exercise 3.2 -

a) Create another C++ script named **test_trajectory2.cpp**

Execute in Shell #2

In [ ]:

```
cd ~/ros2_ws/src/moveit2_scripts/src
touch test_trajectory2.cpp
```

Paste the code below into this file:

test_trajectory2.cpp

In [ ]:

```
#include <moveit/move_group_interface/move_group_interface.h>
#include <moveit/planning_scene_interface/planning_scene_interface.h>

#include <moveit_msgs/msg/display_robot_state.hpp>
#include <moveit_msgs/msg/display_trajectory.hpp>

static const rclcpp::Logger LOGGER = rclcpp::get_logger("move_group_demo");

int main(int argc, char **argv) {
  rclcpp::init(argc, argv);
  rclcpp::NodeOptions node_options;
  node_options.automatically_declare_parameters_from_overrides(true);
  auto move_group_node =
      rclcpp::Node::make_shared("move_group_interface_tutorial", node_options);

  rclcpp::executors::SingleThreadedExecutor executor;
  executor.add_node(move_group_node);
  std::thread([&executor]() { executor.spin(); }).detach();

  static const std::string PLANNING_GROUP = "ur_manipulator";

  moveit::planning_interface::MoveGroupInterface move_group(move_group_node,
                                                            PLANNING_GROUP);

  const moveit::core::JointModelGroup *joint_model_group =
      move_group.getCurrentState()->getJointModelGroup(PLANNING_GROUP);

  RCLCPP_INFO(LOGGER, "Planning frame: %s",
              move_group.getPlanningFrame().c_str());

  RCLCPP_INFO(LOGGER, "End effector link: %s",
              move_group.getEndEffectorLink().c_str());

  RCLCPP_INFO(LOGGER, "Available Planning Groups:");
  std::copy(move_group.getJointModelGroupNames().begin(),
            move_group.getJointModelGroupNames().end(),
            std::ostream_iterator<std::string>(std::cout, ", "));

  geometry_msgs::msg::Pose target_pose1;
  target_pose1.orientation.x = -1.0;
  target_pose1.orientation.y = 0.00;
  target_pose1.orientation.z = 0.00;
```

```
  target_pose1.orientation.w = 0.00;
  target_pose1.position.x = 0.343;
  target_pose1.position.y = 0.132;
  target_pose1.position.z = 0.284;
  move_group.setPoseTarget(target_pose1);

  moveit::planning_interface::MoveGroupInterface::Plan my_plan;

  bool success =
      (move_group.plan(my_plan) == moveit::core::MoveItErrorCode::SUCCESS);

  rclcpp::shutdown();
  return 0;
}
```

Have a closer look at the new code introduced:

In [ ]:

```
geometry_msgs::msg::Pose target_pose1;
target_pose1.orientation.x = -1.0;
target_pose1.orientation.y = 0.00;
target_pose1.orientation.z = 0.00;
target_pose1.orientation.w = 0.00;
target_pose1.position.x = 0.343;
target_pose1.position.y = 0.132;
target_pose1.position.z = 0.284;
move_group.setPoseTarget(target_pose1);
```

As you can see, it's pretty simple. First, define a variable `target_pose1`, and then assign the desired pose values (position and orientation) for the end effector. Finally, set this desired pose as the target for `move_group`.

b) Update the **CMakeLists.txt** file to compile the new script.

Add to CMakeLists.txt

In [ ]:

```
add_executable(test_trajectory2
   src/test_trajectory2.cpp)
target_include_directories(test_trajectory2
   PUBLIC include)
ament_target_dependencies(test_trajectory2
   ament_cmake
         rclcpp
         rclcpp_action
         moveit_core
         moveit_ros_planning_interface
         interactive_markers
         moveit_ros_planning
```

```
        control_msgs)

install(TARGETS
        test_trajectory
        test_trajectory2
    DESTINATION lib/${PROJECT_NAME}
)
```

c) Create a new laucnh file named **test_trajectory2.launch.py**.

Execute in Shell #1

In [ ]:

```
cd ~/ros2_ws/src/moveit2_scripts
cp launch/test_trajectory.launch.py launch/test_trajectory2.launch.py
```

d) Modify the launch file to start the new executable:

In [ ]:

```
moveit_cpp_node = Node(
    name="test_trajectory2",
    package="moveit2_scripts",
    executable="test_trajectory2",
    output="screen",
    parameters=[
        moveit_config.robot_description,
        moveit_config.robot_description_semantic,
        moveit_config.robot_description_kinematics,
        {'use_sim_time': True},
    ],
)
```

e) Recompile your package:

Execute in Shell #3

In [ ]:

```
cd ~/ros2_ws/
colcon build
source install/setup.bash
```

f) Finally, launch the code:

Execute in Shell #3

In [ ]:

```
ros2 launch moveit2_scripts test_trajectory2.launch.py
```

- Notes -

Remember that you need to have MoveIt2 running to run the script. So make sure you start it if you don't have it running already.

Execute in Shell #1

In [ ]:

```
ros2 launch my_moveit_config move_group.launch.py
```

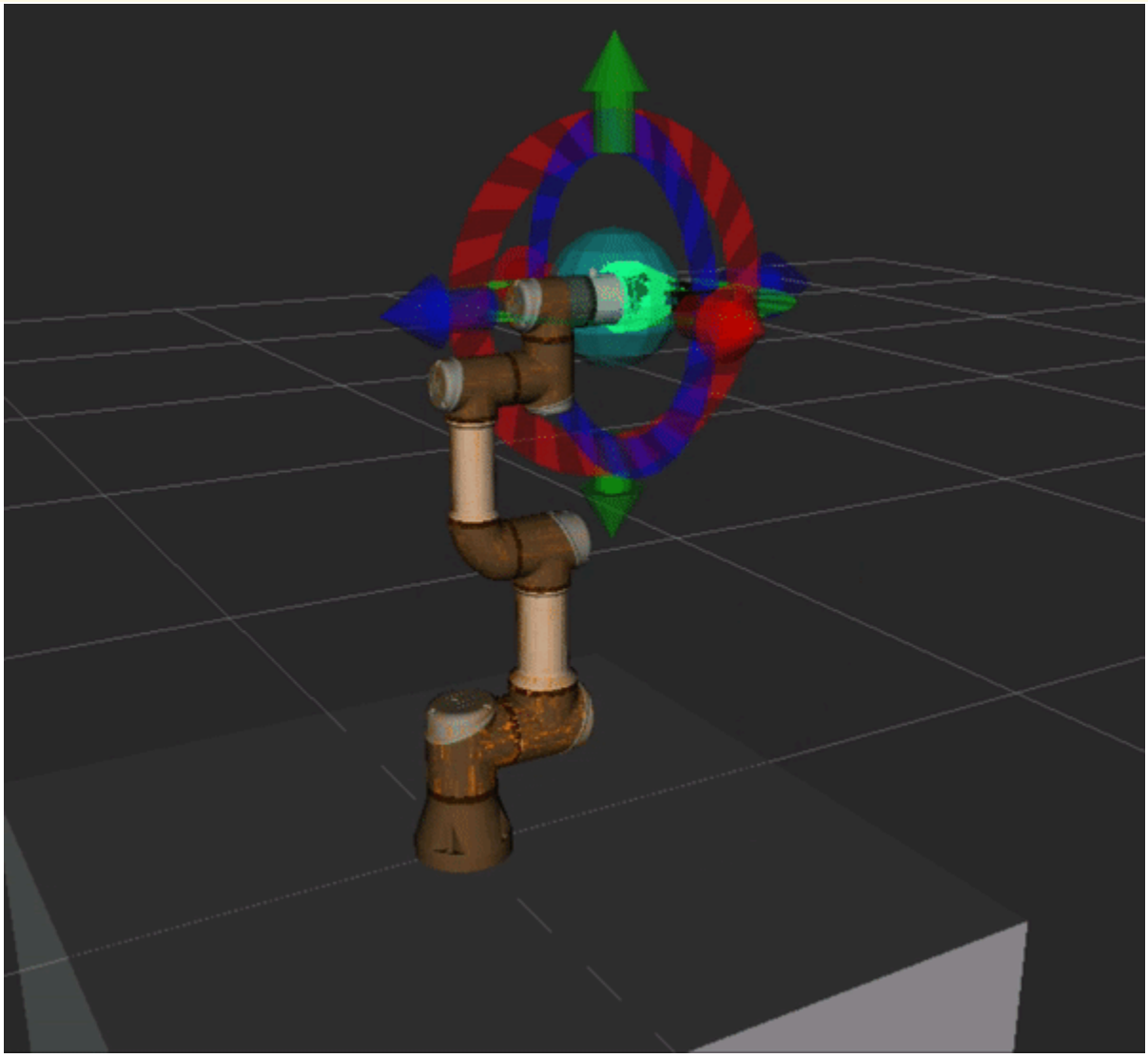
Execute in Shell #2

In [ ]:

```
ros2 launch my_moveit_config moveit_rviz.launch.py
```

- End of Notes -

- Expected Result for Exercise 3.2 -



- End of Expected Result -

- End of Exercise 3.2 -

3.2 Executing a trajectory

So, at this point, you've seen some methods that allow you to plan a trajectory with C++ code. But what about executing this trajectory?

It's very simple. You need to call the `execute (plan)` function from the planning group to execute a trajectory. For instance, given a plan `my_plan`, you would execute the planned motion like this:

In [ ]:

```
move_group.execute(my_plan_arm);
```

By executing this line of code, you will be telling your robot to **Execute** the trajectory stored in the `my_plan` variable.
Also, there's the option to **Plan & Execute** a motion, just as the GUI provides.

In [ ]:

```
move_group.move();
```

Try this in an exercise!

- Exercise 3.3 -

Modify the C++ programs created in the previous exercises to execute the trajectory.

Modified test_trajectory.cpp

In [ ]:

```
#include <moveit/move_group_interface/move_group_interface.h>
#include <moveit/planning_scene_interface/planning_scene_interface.h>

#include <moveit_msgs/msg/display_robot_state.hpp>
#include <moveit_msgs/msg/display_trajectory.hpp>

static const rclcpp::Logger LOGGER = rclcpp::get_logger("move_group_demo");
static const std::string PLANNING_GROUP = "ur_manipulator";

int main(int argc, char **argv) {
  rclcpp::init(argc, argv);
  rclcpp::NodeOptions node_options;
  node_options.automatically_declare_parameters_from_overrides(true);
  auto move_group_node =
      rclcpp::Node::make_shared("move_group_interface_tutorial", node_options);

  rclcpp::executors::SingleThreadedExecutor executor;
  executor.add_node(move_group_node);
  std::thread([&executor]() { executor.spin(); }).detach();

  moveit::planning_interface::MoveGroupInterface move_group(move_group_node,
                                                            PLANNING_GROUP);

  const moveit::core::JointModelGroup *joint_model_group =
```

```
    move_group.getCurrentState()->getJointModelGroup(PLANNING_GROUP);

  RCLCPP_INFO(LOGGER, "Planning frame: %s",
              move_group.getPlanningFrame().c_str());

  RCLCPP_INFO(LOGGER, "End effector link: %s",
              move_group.getEndEffectorLink().c_str());

  RCLCPP_INFO(LOGGER, "Available Planning Groups:");
  std::copy(move_group.getJointModelGroupNames().begin(),
            move_group.getJointModelGroupNames().end(),
            std::ostream_iterator<std::string>(std::cout, ", "));

  moveit::core::RobotStatePtr current_state = move_group.getCurrentState(10);

  std::vector<double> joint_group_positions;
  current_state->copyJointGroupPositions(joint_model_group,
                                         joint_group_positions);

  //joint_group_positions[0] = 0.00;  // Shoulder Pan
  joint_group_positions[1] = -2.50;  // Shoulder Lift
  joint_group_positions[2] = 1.50;  // Elbow
  joint_group_positions[3] = -1.50;  // Wrist 1
  joint_group_positions[4] = -1.55;  // Wrist 2
  //joint_group_positions[5] = 0.00;  // Wrist 3
  move_group.setJointValueTarget(joint_group_positions);

  moveit::planning_interface::MoveGroupInterface::Plan my_plan;

  bool success =
      (move_group.plan(my_plan) == moveit::core::MoveItErrorCode::SUCCESS);

  move_group.execute(my_plan);

  rclcpp::shutdown();
  return 0;
}
```

Recompile your package and test it:

Execute in Shell #4

In [ ]:

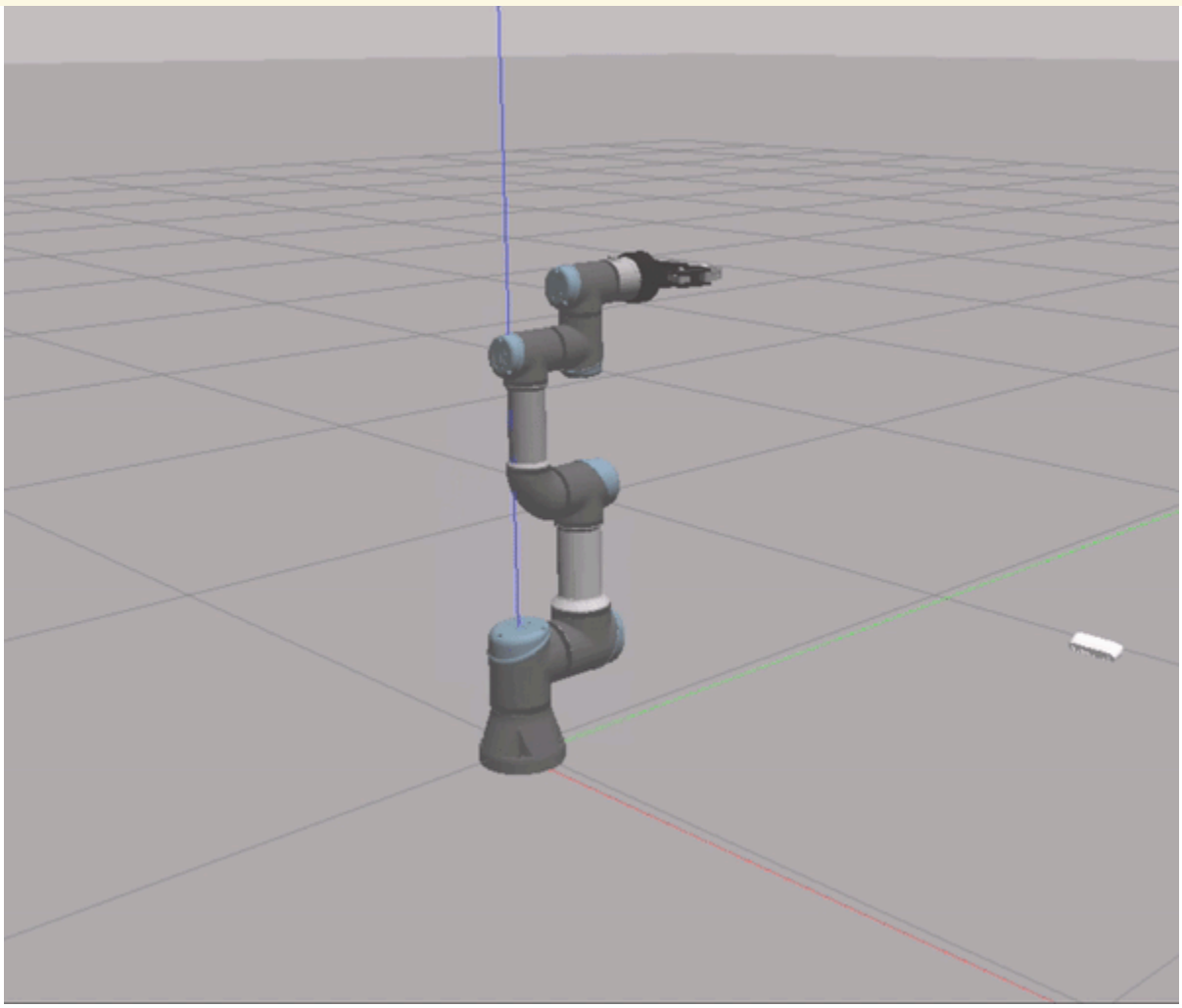
```
cd ~/ros2_ws
colcon build
source install/setup.bash
```

Execute in Shell #4

In [ ]:

```
ros2 launch moveit2_scripts test_trajectory.launch.py
```

- Expected Result for Exercise 3.3 -



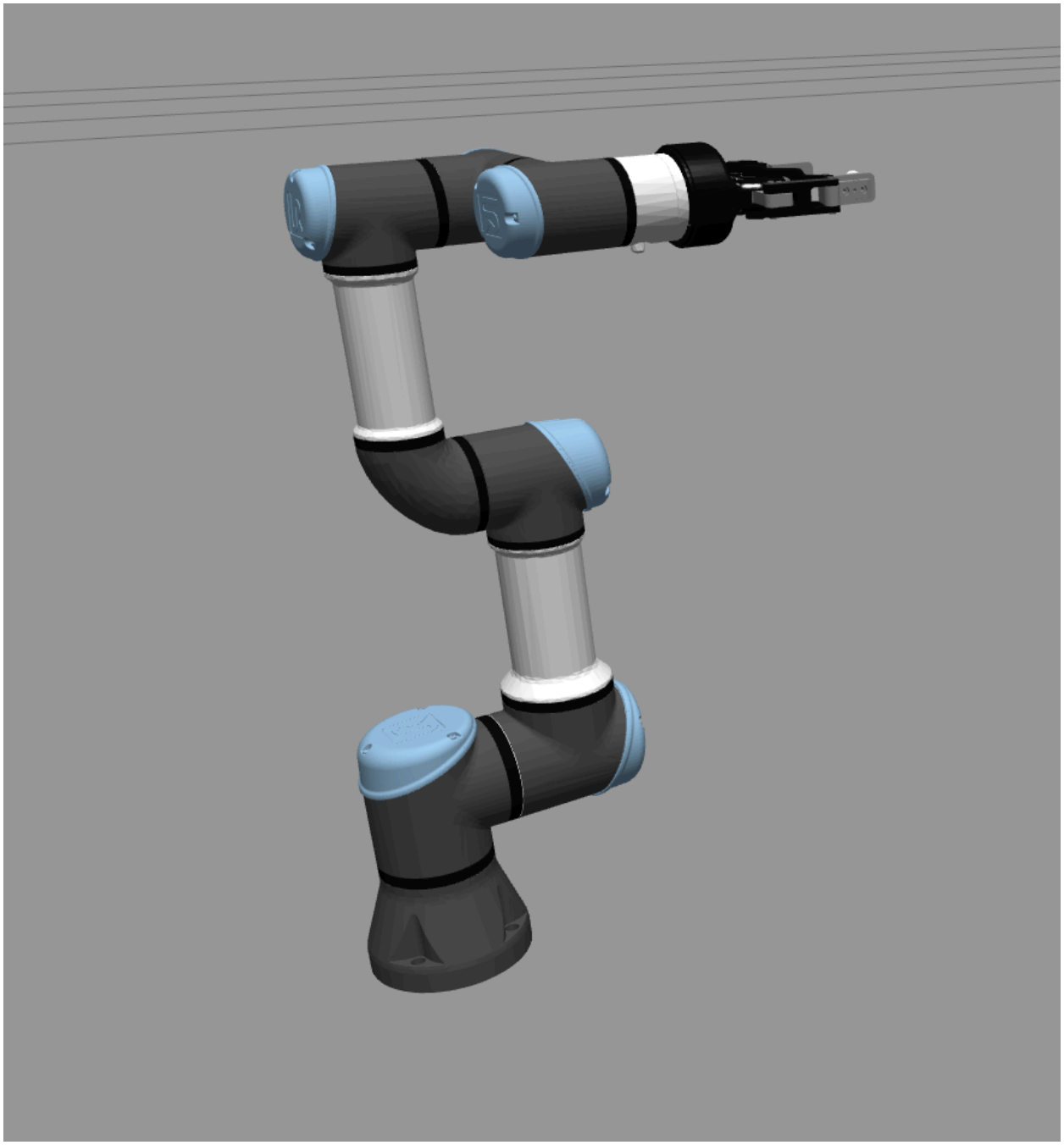
- End of Expected Result for Exercise 3.3 -

- End of Exercise 3.3 -

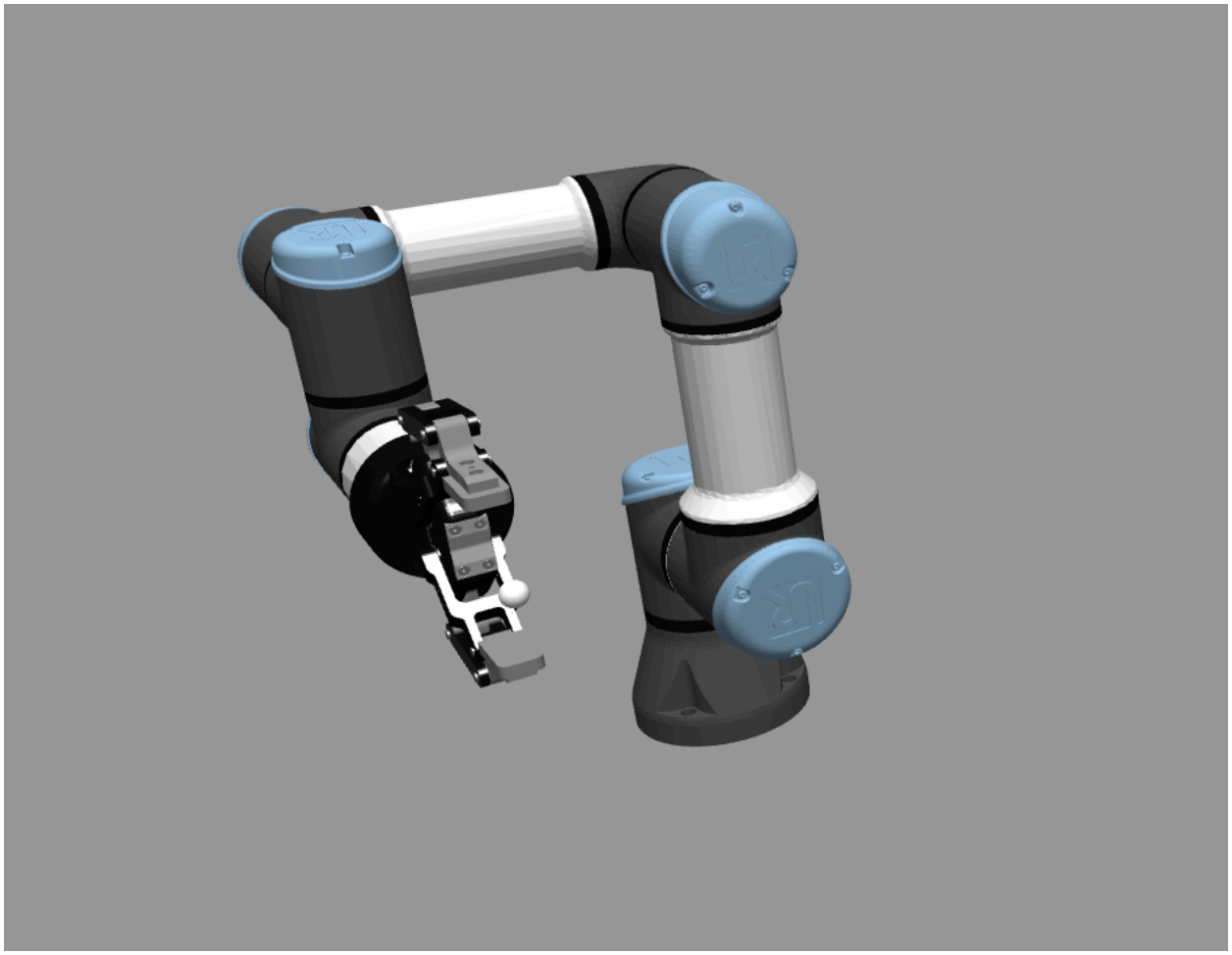
- Exercise 3.4 -

Create a new program in a C++ script named **test_trajectory3.cpp**. This new program plans and executes two different trajectories. The workflow will be as follows:

- First, the program will tell MoveIt2 to plan and execute a trajectory for the robotic arm to achieve the following configuration:

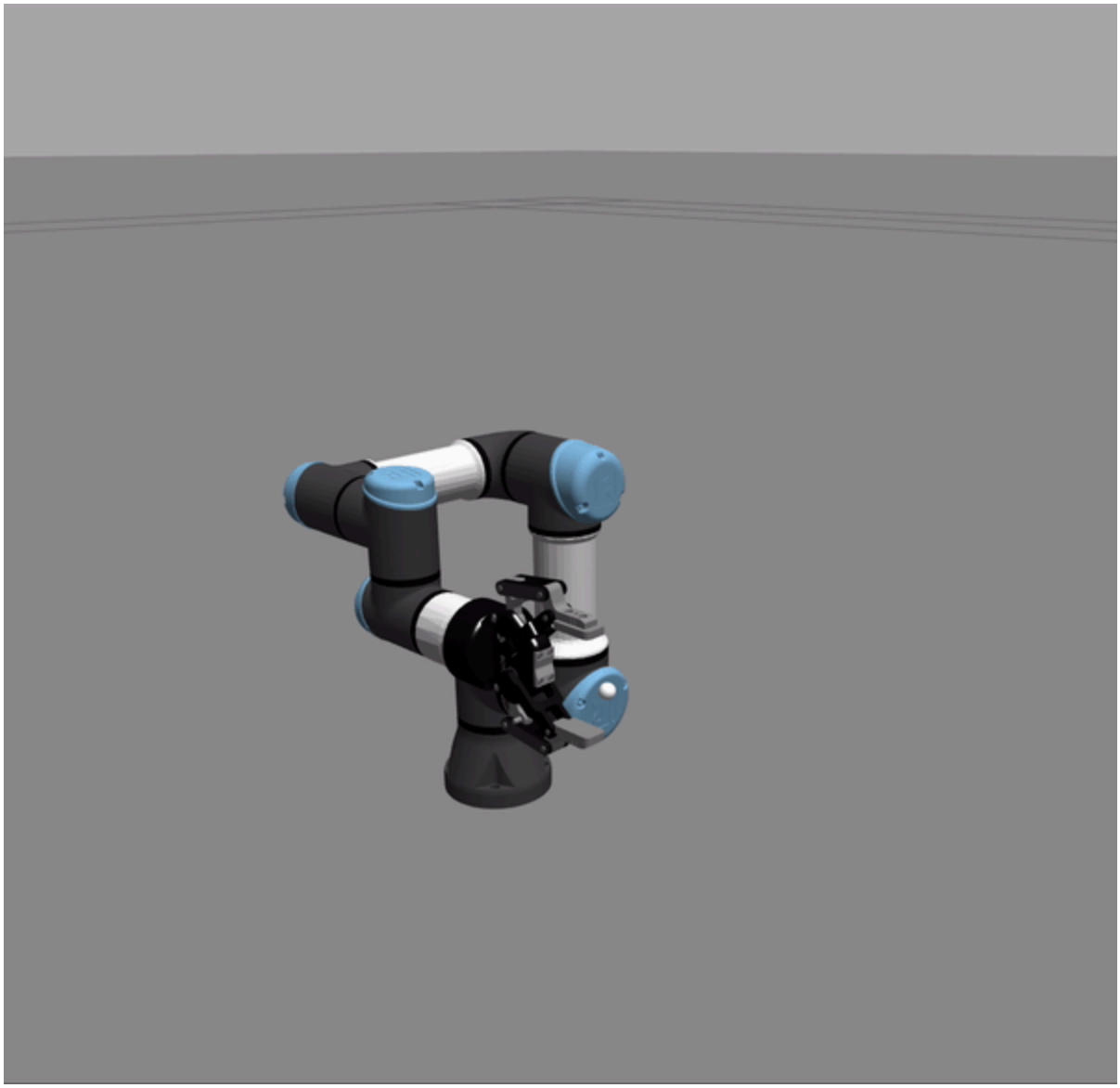


- After the first motion finishes, the robot will plan and execute the second motion, this time to achieve the following joint configuration:



- End of Exercise 3.4 -

- Expected Result for Exercise 3.4 -



- End of Expected Result -

- Solution for Exercise 3.4 -

test_trajectory3.cpp

In [ ]:

```
#include <moveit/move_group_interface/move_group_interface.h>
#include <moveit/planning_scene_interface/planning_scene_interface.h>

#include <moveit_msgs/msg/display_robot_state.hpp>
#include <moveit_msgs/msg/display_trajectory.hpp>

static const rclcpp::Logger LOGGER = rclcpp::get_logger("move_group_demo");
static const std::string PLANNING_GROUP = "ur_manipulator";

int main(int argc, char **argv) {
  rclcpp::init(argc, argv);
```

```
rclcpp::NodeOptions node_options;
node_options.automatically_declare_parameters_from_overrides(true);
auto move_group_node =
    rclcpp::Node::make_shared("move_group_interface_tutorial", node_options);

rclcpp::executors::SingleThreadedExecutor executor;
executor.add_node(move_group_node);
std::thread([&executor]() { executor.spin(); }).detach();

moveit::planning_interface::MoveGroupInterface move_group(move_group_node,
                                                          PLANNING_GROUP);

const moveit::core::JointModelGroup *joint_model_group =
    move_group.getCurrentState()->getJointModelGroup(PLANNING_GROUP);

RCLCPP_INFO(LOGGER, "Planning frame: %s",
            move_group.getPlanningFrame().c_str());

RCLCPP_INFO(LOGGER, "End effector link: %s",
            move_group.getEndEffectorLink().c_str());

RCLCPP_INFO(LOGGER, "Available Planning Groups:");
std::copy(move_group.getJointModelGroupNames().begin(),
          move_group.getJointModelGroupNames().end(),
          std::ostream_iterator<std::string>(std::cout, ", "));

moveit::core::RobotStatePtr current_state = move_group.getCurrentState(10);

std::vector<double> joint_group_positions;
current_state->copyJointGroupPositions(joint_model_group,
                                       joint_group_positions);

joint_group_positions[0] = 0.00;  // Shoulder Pan
joint_group_positions[1] = -1.57; // Shoulder Lift
joint_group_positions[2] = 0.00;  // Elbow
joint_group_positions[3] = 0.00;  // Wrist 1
joint_group_positions[4] = 0.00;  // Wrist 2
// joint_group_positions[5] = 0.00;  // Wrist 3
move_group.setJointValueTarget(joint_group_positions);

moveit::planning_interface::MoveGroupInterface::Plan my_plan;

bool success =
    (move_group.plan(my_plan) == moveit::core::MoveItErrorCode::SUCCESS);

// Execute
```

```
  RCLCPP_INFO(LOGGER, "Going to Position 1");
  move_group.execute(my_plan);

  joint_group_positions[0] = -1.57; // Shoulder Pan
  joint_group_positions[1] = -1.57; // Shoulder Lift
  joint_group_positions[2] = 1.57;  // Elbow
  joint_group_positions[3] = 0.00;  // Wrist 1
  joint_group_positions[4] = 0.00;  // Wrist 2
  joint_group_positions[5] = 1.57;  // Wrist 3
  move_group.setJointValueTarget(joint_group_positions);

  success =
     (move_group.plan(my_plan) == moveit::core::MoveItErrorCode::SUCCESS);

  // Execute
  RCLCPP_INFO(LOGGER, "Going to Position 2");
  move_group.execute(my_plan);

  rclcpp::shutdown();
  return 0;
}
```

- End Solution for Exercise 3.4 -

3.3 How to get your robot arms positioned fast and easy

Getting the End-Effector pose or joints as you want just with numbers and testing is tedious and inefficient. For this purpose, make use of a tool to allow you to move the robotic arm fast and intuitively:

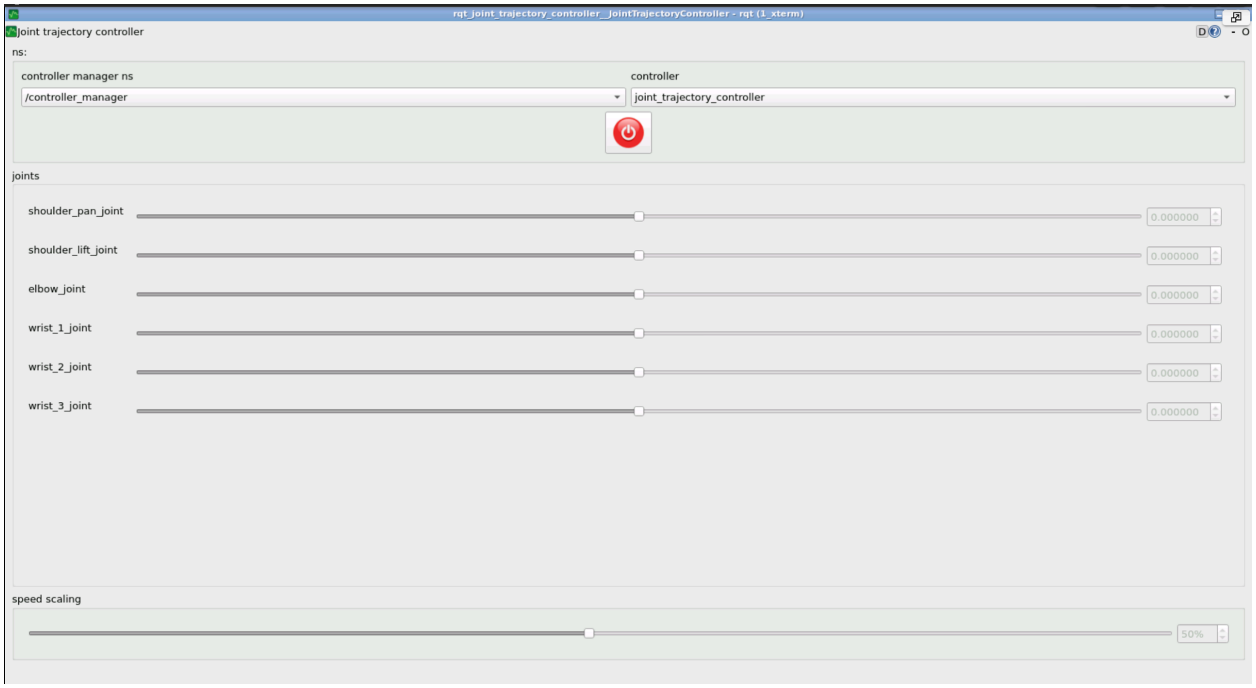
Execute in Shell #1

In [ ]:

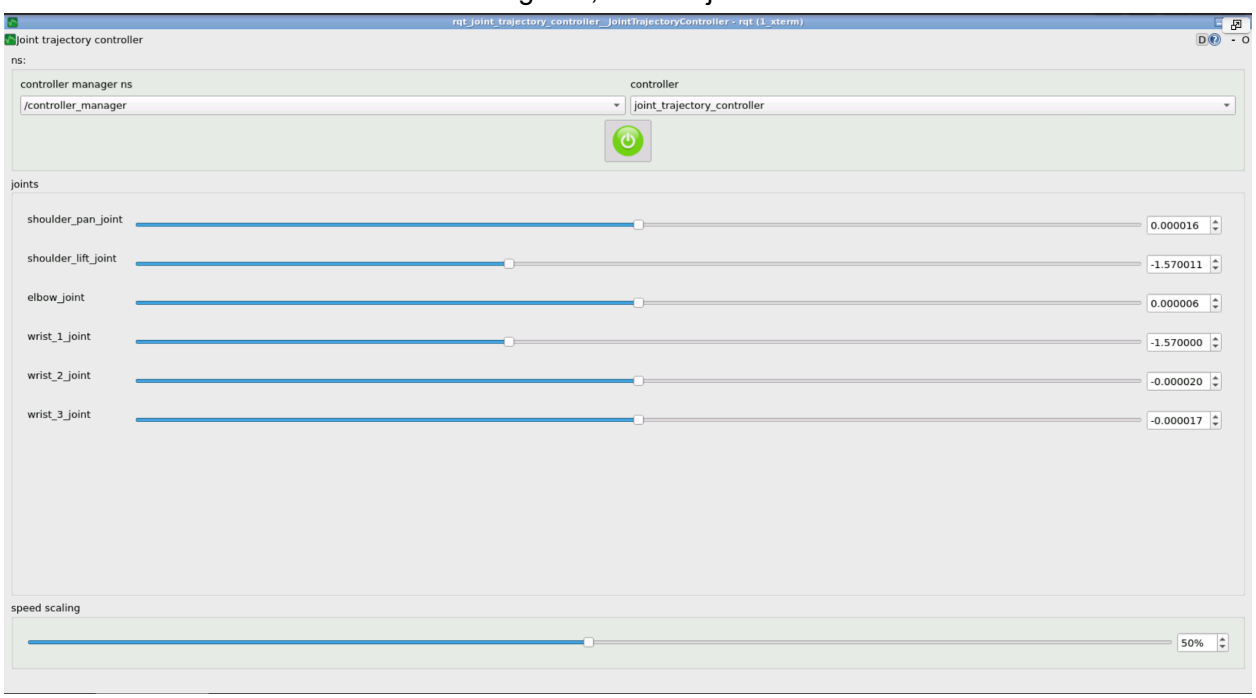
```
ros2 run rqt_joint_trajectory_controller rqt_joint_trajectory_controller
```

This command launches the `rqt_joint_trajectory_controller` node, which provides a GUI to move the Joints Freely (remember to open the Graphical Tools screen, pressing on the small screen button in the IDE).

Now, set the **controller manager ns** and **controller** variables with the options shown in the figure below and then hit the red ON/OFF button:



You should now see how the button turns green, and the joint slides become available.



You can now control each joint of the arm freely. Play with the slides and move the robot to different configurations.
Also, have a quick look at a couple of methods that allow you to get the joint and end-effector positions easily.

1. Get the end-effector pose

You can use the command line to get the position of the end-effector relative to the robot base frame (or any other robot frame). For this, you will use the **transforms** (TF) of the robot.

Execute in Shell #1

- WARNING - To avoid issues with different ROS versions, close the web shell to trigger it to restart with a clean shell. For that, click on the **cross** on the top right corner of the shell you want to restart.

In [ ]:

```
ros2 run tf2_ros tf2_echo base_link tool0
```

Expected Output

In [ ]:

```
At time 1350.570000000
- Translation: [0.457, 0.223, 0.067]
- Rotation: in Quaternion [-0.000, 0.707, 0.707, -0.000]
At time 1351.550000000
```

2. Get the joint positions

Additionally, you can get the current joint positions by checking the **/joint_states** topic. The **Joint State Controller** provides the joint states of the robot.

Execute in Shell #2

In [ ]:

```
ros2 topic echo /joint_states
```

Expected Output

In [ ]:

```
header:
  stamp:
    sec: 221
    nanosec: 658000000
  frame_id: ''
name:
- shoulder_pan_joint
- shoulder_lift_joint
- elbow_joint
- wrist_2_joint
- wrist_3_joint
- wrist_1_joint
- robotiq_85_left_knuckle_joint
position:
- 1.5444652730067787e-05
- -1.5700090008239351
- 4.8789799471649076e-06
- -1.938054995598293e-05
- -1.693994438589641e-05
- -1.5699996624648191
- 7.193142447903256e-06
```

```
velocity:
- -0.000373574662565984
- 0.001999174211441514
- -0.001199828123609586
- 0.0004902811418939408
- -2.74005972728061e-05
- 0.0003375325463356836
- -0.004076884668536682
effort:
- .nan
- .nan
- .nan
- .nan
- .nan
- .nan
- .nan
```

3.4 Controlling the gripper

As you should have already noted, the UR3e Arm has a gripper mounted on its end-effector. So far, you've only been working with the robot arm, so get the gripper a little bit involved!

As you may have already noticed, the gripper is controlled by a **GripperCommand** action server.

In [ ]:

```
/gripper_controller/gripper_cmd
```

So, to control the gripper, you will have to send a goal to its action server.

Execute in Shell #2

OPEN GRIPPER

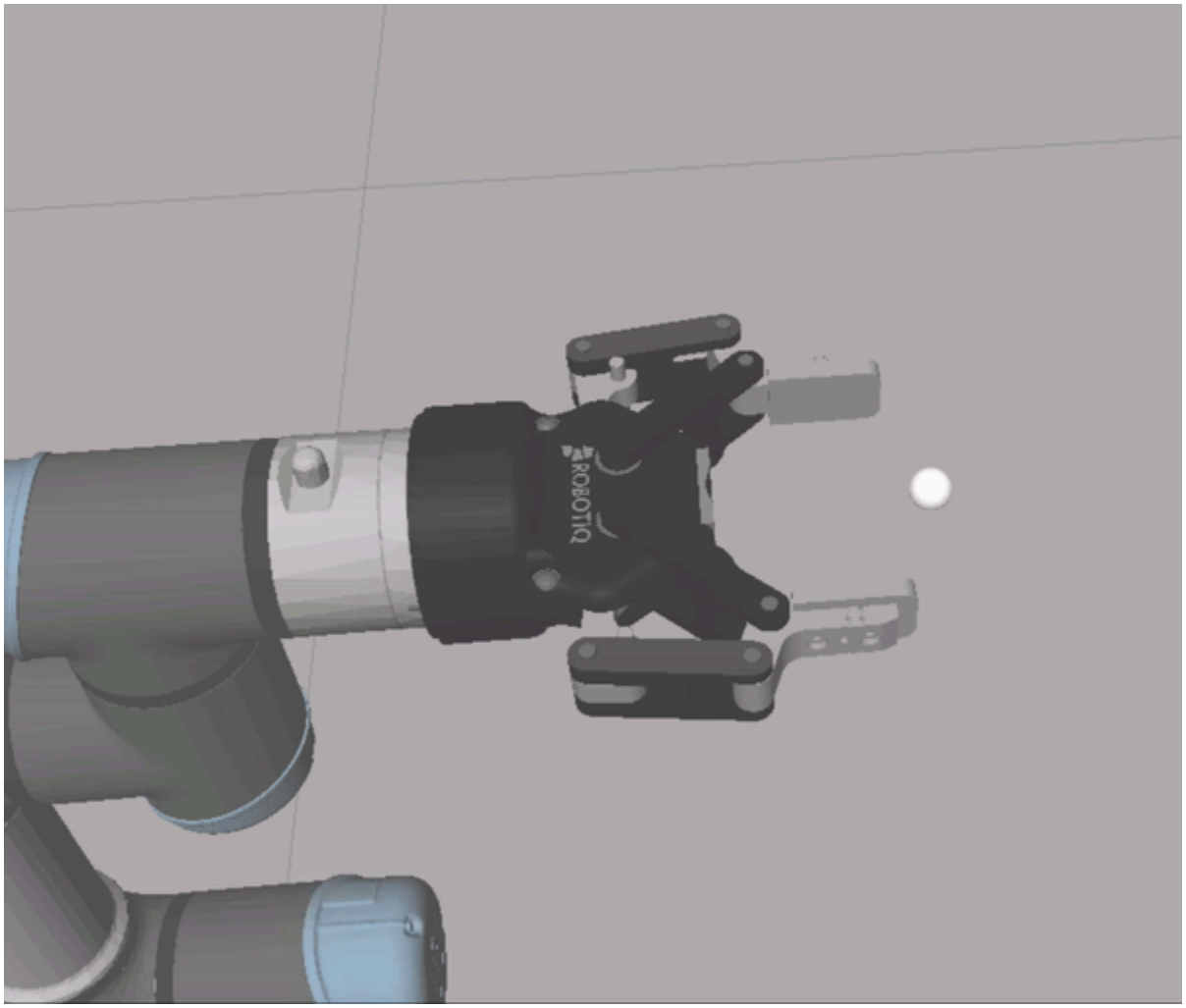
In [ ]:

```
ros2 action send_goal /gripper_controller/gripper_cmd control_msgs/action/GripperCommand
"command:
  position: 0.0
  max_effort: 0.0
"
```

CLOSE GRIPPER

In [ ]:

```
ros2 action send_goal /gripper_controller/gripper_cmd control_msgs/action/GripperCommand
"command:
  position: 0.8
  max_effort: 0.0
"
```



3.5 Controlling the gripper with MoveIt2

You have already learnt how to control the aram using C++ program. But is it possible to do the sasme with the gripper? Let's find the answer in the next exercise!

- Exercise 3.5 -

Let's have a quick look at the code that you will be executing:

test_gripper.cpp

In [ ]:

```
#include <moveit/move_group_interface/move_group_interface.h>
#include <moveit/planning_scene_interface/planning_scene_interface.h>

#include <moveit_msgs/msg/display_robot_state.hpp>
#include <moveit_msgs/msg/display_trajectory.hpp>

#include <chrono>
#include <thread>
```

```
static const rclcpp::Logger LOGGER = rclcpp::get_logger("move_group_demo");
static const std::string PLANNING_GROUP_GRIPPER = "gripper";

int main(int argc, char **argv) {
  rclcpp::init(argc, argv);
  rclcpp::NodeOptions node_options;
  node_options.automatically_declare_parameters_from_overrides(true);
  auto move_group_node =
      rclcpp::Node::make_shared("move_group_interface_tutorial", node_options);

  rclcpp::executors::SingleThreadedExecutor executor;
  executor.add_node(move_group_node);
  std::thread([&executor]() { executor.spin(); }).detach();

  moveit::planning_interface::MoveGroupInterface move_group_gripper(
      move_group_node, PLANNING_GROUP_GRIPPER);

  const moveit::core::JointModelGroup *joint_model_group_gripper =
      move_group_gripper.getCurrentState()->getJointModelGroup(
          PLANNING_GROUP_GRIPPER);

  moveit::core::RobotStatePtr current_state_gripper =
      move_group_gripper.getCurrentState(10);

  std::vector<double> joint_group_positions_gripper;
  current_state_gripper->copyJointGroupPositions(joint_model_group_gripper,
                                                 joint_group_positions_gripper);

  // Close Gripper

  RCLCPP_INFO(LOGGER, "Close Gripper!");

  joint_group_positions_gripper[2] = 0.55;

  move_group_gripper.setJointValueTarget(joint_group_positions_gripper);

  moveit::planning_interface::MoveGroupInterface::Plan my_plan_gripper;
  bool success_gripper = (move_group_gripper.plan(my_plan_gripper) ==
                          moveit::core::MoveItErrorCode::SUCCESS);

  move_group_gripper.execute(my_plan_gripper);

  // Sleep for some seconds
  std::this_thread::sleep_for(std::chrono::milliseconds(3000));

  // Open Gripper
```

```
  RCLCPP_INFO(LOGGER, "Open Gripper!");

  move_group_gripper.setNamedTarget("gripper_open");

  success_gripper = (move_group_gripper.plan(my_plan_gripper) ==
                     moveit::core::MoveItErrorCode::SUCCESS);

  move_group_gripper.execute(my_plan_gripper);

  rclcpp::shutdown();
  return 0;
}
```

Let's have a closer look at the new code introduced!
First of all we define a static variables for the **planning group of the gripper**, which is named `gripper`:

In [ ]:

```
static const std::string PLANNING_GROUP_GRIPPER = "gripper";
```

Next we create the `MoveGroupInterface` for the gripper:

In [ ]:

```
moveit::planning_interface::MoveGroupInterface move_group_gripper(move_group_node,
                                                  PLANNING_GROUP_GRIPPER);
```

As you can see, we initialize a `MoveGroupInterface` named `move_group_gripper` for the gripper planning group. This interface will allow you to call the move group node capabilities from a C++ program.
Next we create a `JointModelGroup` pointer:

In [ ]:

```
const moveit::core::JointModelGroup *joint_model_group_gripper =
  move_group.getCurrentState()->getJointModelGroup(PLANNING_GROUP_GRIPPER);
```

Pointers are often used to refer to the planning group for improved performance. This `JointModelGroup` pointer will contain all the information related to the planning group.
Next we get the `current state` of the gripper:

In [ ]:

```
moveit::core::RobotStatePtr current_state_gripper =
    move_group_gripper.getCurrentState(10);

std::vector<double> joint_group_positions_gripper;
  current_state_gripper->copyJointGroupPositions(joint_model_group_gripper,
                                                 joint_group_positions_gripper);
```

What we are doing here is to get the current state of the gripper. Then we copy the values of the variable `joint_model_group_gripper` (which contains the position value of each joint of the gripper) into the vector `joint_group_positions_gripper`.
Finally we plan a motion by changing the position values of the joints:

In [ ]:

```
joint_group_positions_gripper[2] = 0.55;

move_group_gripper.setJointValueTarget(joint_group_positions_gripper);

moveit::planning_interface::MoveGroupInterface::Plan my_plan_gripper;
bool success_gripper = (move_group_gripper.plan(my_plan_gripper) ==
                        moveit::core::MoveItErrorCode::SUCCESS);

move_group_gripper.execute(my_plan_gripper);
```

Here we are doing a couple of important things:

- First, we set the desired position values of the joints. In this case, we are setting the joint value for closing the gripper.
- Second, we set these new values as the new target using the `setJointValueTarget` method.
- Finally, we call the `plan()` method of the move group interface to plan the trajectory.

After closing the gripper, we generate another plan in order to open the gripper:

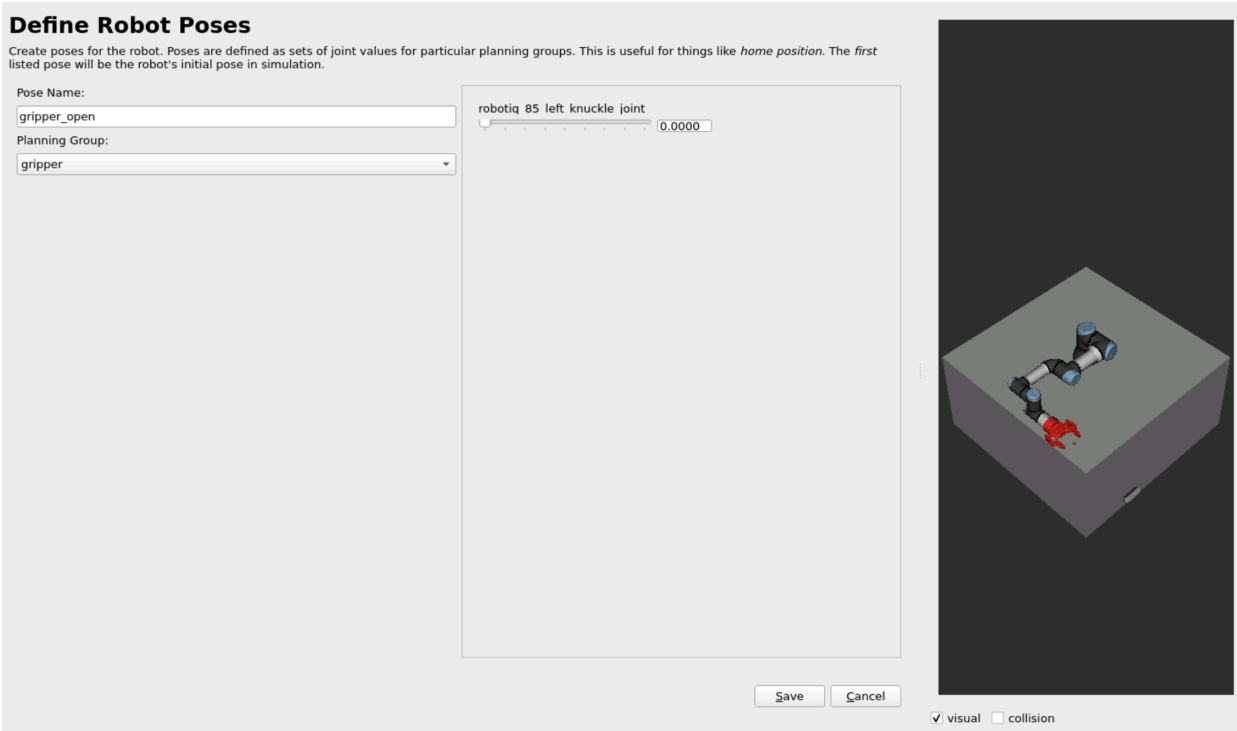
In [ ]:

```
move_group_gripper.setNamedTarget("gripper_open");

success_gripper = (move_group_gripper.plan(my_plan_gripper) ==
                   moveit::core::MoveItErrorCode::SUCCESS);

move_group_gripper.execute(my_plan_gripper);
```

Note that in this case, we are setting the new target using the method `setNamedTarget`. This method allows to set a target based on the predefined poses you added when creating the MoveIt2 package (like the one in the image below).



Let's create the files and launch them

a) Inside your **ros2_ws**, create a new package named **moveit2_scripts** with some dependencies.
a) Inside the **moveit2_scripts** package, create a new file named **test_gripper.cpp** and copy the code you've just seen above inside this file.

In [ ]:

```
cd ~/ros2_ws/src/moveit2_scripts/src
touch test_gripper.cpp
```

b) Inside the package, also create a launch file to launch the program, called **test_gripper.launch.py**.

Execute in Shell #1

In [ ]:

```
cd ~/ros2_ws/src/moveit2_scripts
mkdir launch
touch launch/test_gripper.launch.py
```

Paste the code below into this launch file:

test_gripper.launch.py

In [ ]:

```
import os
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.actions import ExecuteProcess
from ament_index_python.packages import get_package_share_directory
```

```python
from moveit_configs_utils import MoveItConfigsBuilder


def generate_launch_description():
    moveit_config = MoveItConfigsBuilder("name",
package_name="my_moveit_config").to_moveit_configs()

    # MoveItCpp demo executable
    moveit_cpp_node = Node(
        name="test_gripper",
        package="moveit2_scripts",
        executable="test_gripper",
        output="screen",
        parameters=[
            moveit_config.robot_description,
            moveit_config.robot_description_semantic,
            moveit_config.robot_description_kinematics,
            {'use_sim_time': True},
        ],
    )

    return LaunchDescription(
        [moveit_cpp_node]
    )
```

c) Now, update the **CMakeLists.txt** file to compile everything. First, open the file and add the following lines to the code:

Add to CMakeLists.txt

In [ ]:

```
add_executable(test_gripper
    src/test_gripper.cpp)
target_include_directories(test_gripper
    PUBLIC include)
ament_target_dependencies(test_gripper
    ament_cmake
            rclcpp
            rclcpp_action
            moveit_core
            moveit_ros_planning_interface
            interactive_markers
            moveit_ros_planning
            control_msgs)

# Install the executable
install(TARGETS
            test_gripper
```

```
    DESTINATION lib/${PROJECT_NAME}
)

# Install the launch file
install(DIRECTORY
          launch
    DESTINATION share/${PROJECT_NAME}
)
```

d) Great! Let's now compile our workspace.

Execute in Shell #1

In [ ]:

```
cd ~/ros2_ws
colcon build --packages-select moveit2_scripts
source install/setup.bash
```

e) Now test your new program! First, you'll need to launch the MoveIt2 RVIZ environment.

- Reminder -

Remember that before being able to use the move group interface, you need to start the move group node and, optionally, moveit rviz for visualization purposes.

Execute in Shell #1

In [ ]:

```
ros2 launch my_moveit_config move_group.launch.py
```

Execute in Shell #2

In [ ]:

```
ros2 launch my_moveit_config moveit_rviz.launch.py
```

- End of Reminder -

f) Now, run your code:

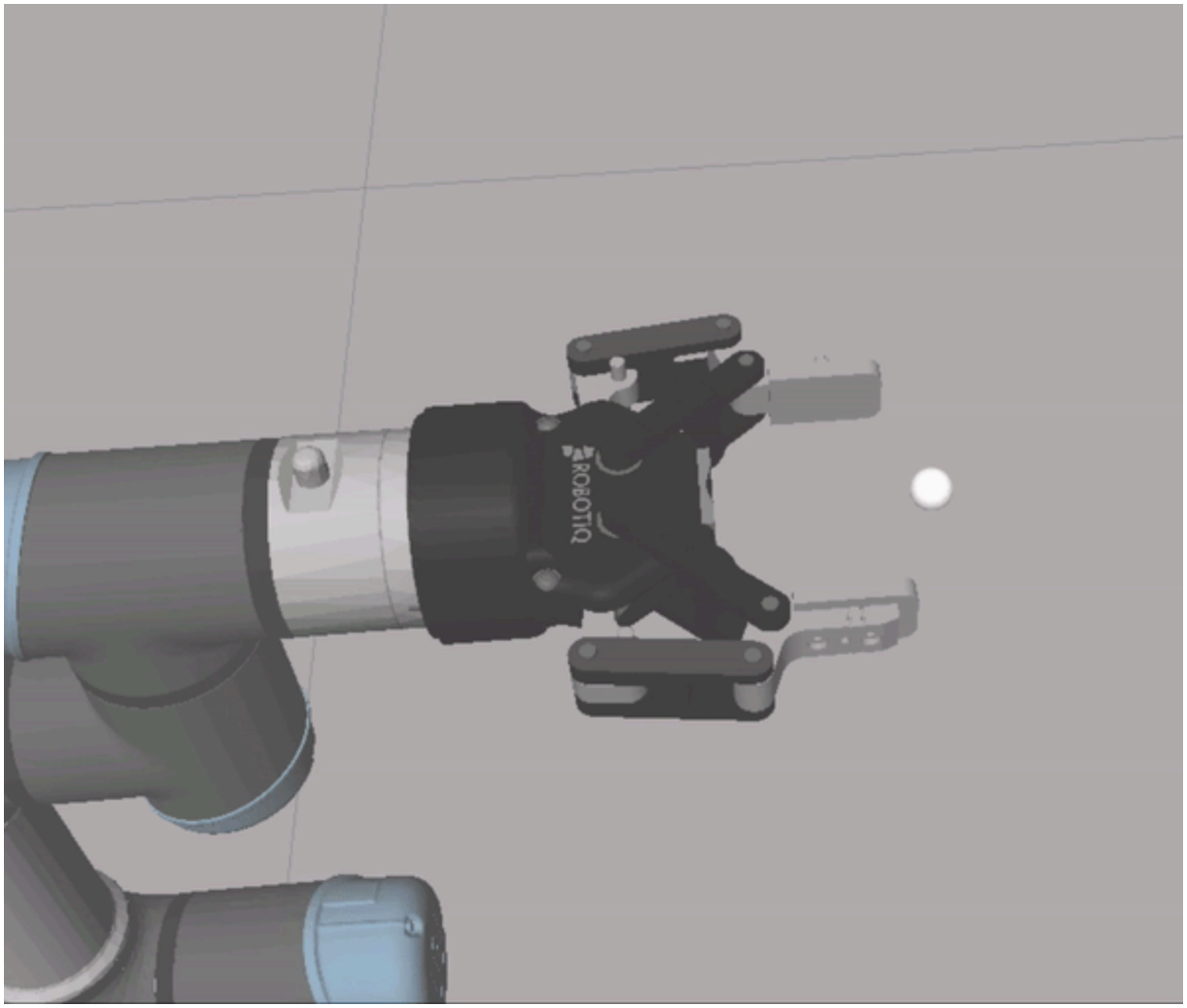
Execute in Shell #3

In [ ]:

```
ros2 launch moveit2_scripts test_gripper.launch.py
```

You should now see now the gripper close and open.

- Expected Result for Exercise 3.5 -



- End of Expected Result -

- End of Excersie 3.5 -

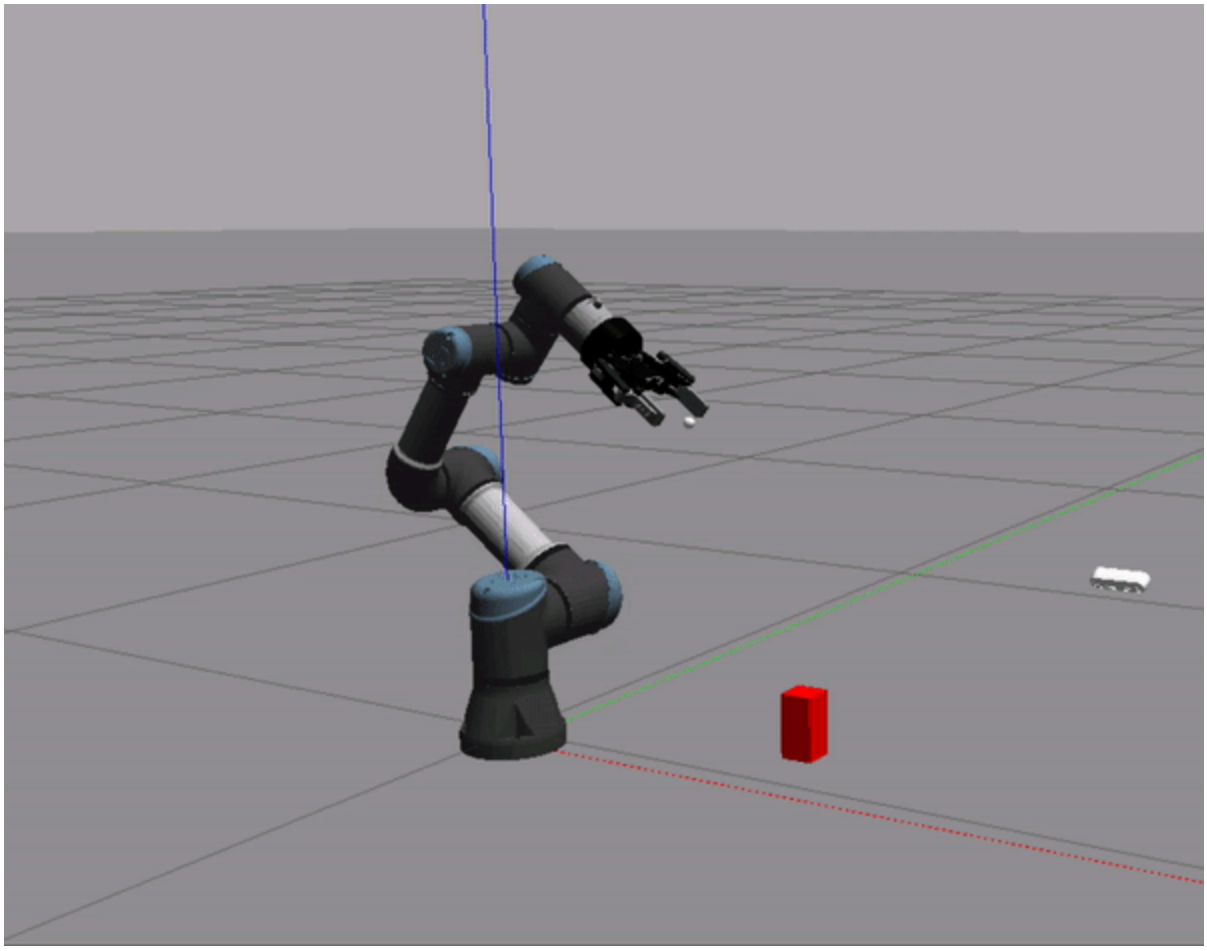
3.6 Approach & Retreat

Now that you have learned how to control the robotic arm and the gripper, let's try to put everything together! For that, you will have to complete an exercise that performs the `approach & retreat` motions.

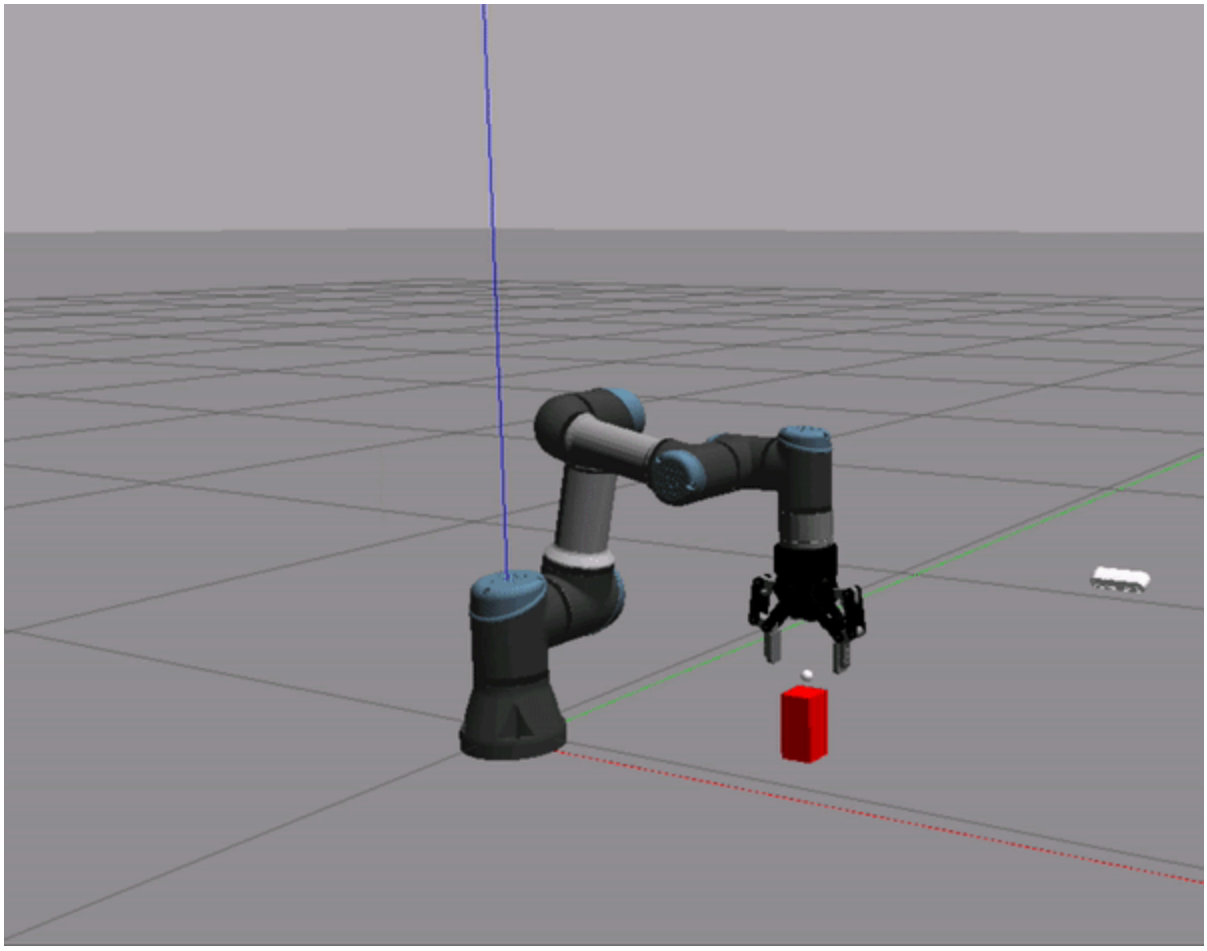
In manipulation, when you want to pick an object from the environment, you don't send the end-effector directly to the pose of the object, as the gripper might end up colliding with it. Instead, you send the end-effector to a pose **near** the object. Then, you execute the **approach** motion in order to get close enoguh to the object to pick it. Once you have picked the object, you execute the **retreat** motion to go back to the previous position.

The full sequence would look something like this:

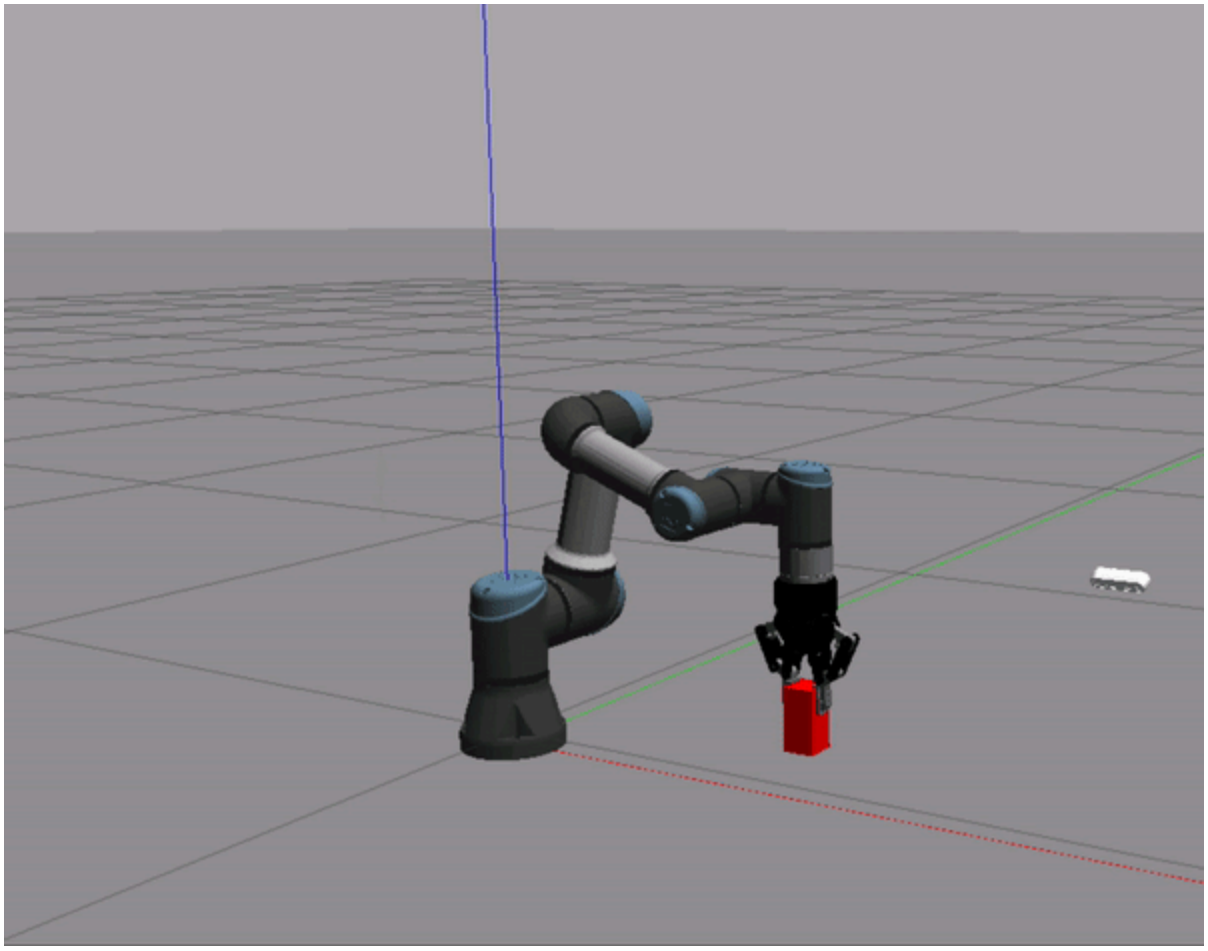
1. Move the arm to a pose near the object:



2. Approach:



3. Retreat:



Alright! So let's try to implement the **approach & retreat** sequence in the following exercise.

- Exercise 3.6 -

a) Before getting started, we need to spawn an object into the scene. For that, create a new file named `grasp_box.urdf` inside `~/ros2_ws/src`:

Execute in Shell

In [ ]:

```
cd ~/ros2_ws/src
touch grasp_box.urdf
```

grasp_box.urdf

In [ ]:

```
<robot name="grasp_box">

    <!-- Colours for RVIZ for geometric elements -->
    <material name="blue">
        <color rgba="0 0 0.8 1"/>
    </material>
    <material name="red">
        <color rgba="0.8 0 0 1"/>
    </material>
```

```
    <material name="green">
        <color rgba="0 0.8 0 1"/>
    </material>
    <material name="grey">
        <color rgba="0.75 0.75 0.75 1"/>
    </material>
    <material name="white">
        <color rgba="1.0 1.0 1.0 1"/>
    </material>
    <material name="black">
        <color rgba="0 0 0 1"/>
    </material>


            <!-- * * * Link Definitions * * * -->

    <link name="grasp_box_base_link">
                <inertial>
            <origin xyz="0 0 0" rpy="0 0 0"/>
            <mass value="0.5" />
            <inertia ixx="0.000133333333333" ixy="0.0" ixz="0.0" iyy="0.000133333333333" iyz="0.0"
izz="0.000133333333333"/>
        </inertial>
        <collision>
            <origin xyz="0 0 0" rpy="0 0 0"/>
            <geometry>
                <box size="0.04 0.04 0.08"/>
            </geometry>
        </collision>
        <visual>
            <origin rpy="0.0 0 0" xyz="0 0 0"/>
            <geometry>
                <box size="0.04 0.04 0.08"/>
            </geometry>
            <material name="red"/>
        </visual>
    </link>

    <gazebo reference="grasp_box_base_link">
        <kp>100000.0</kp>
        <kd>100.0</kd>
        <mu1>0.25</mu1>
        <mu2>0.25</mu2>
        <material>Gazebo/Red</material>
    </gazebo>
```

</robot>

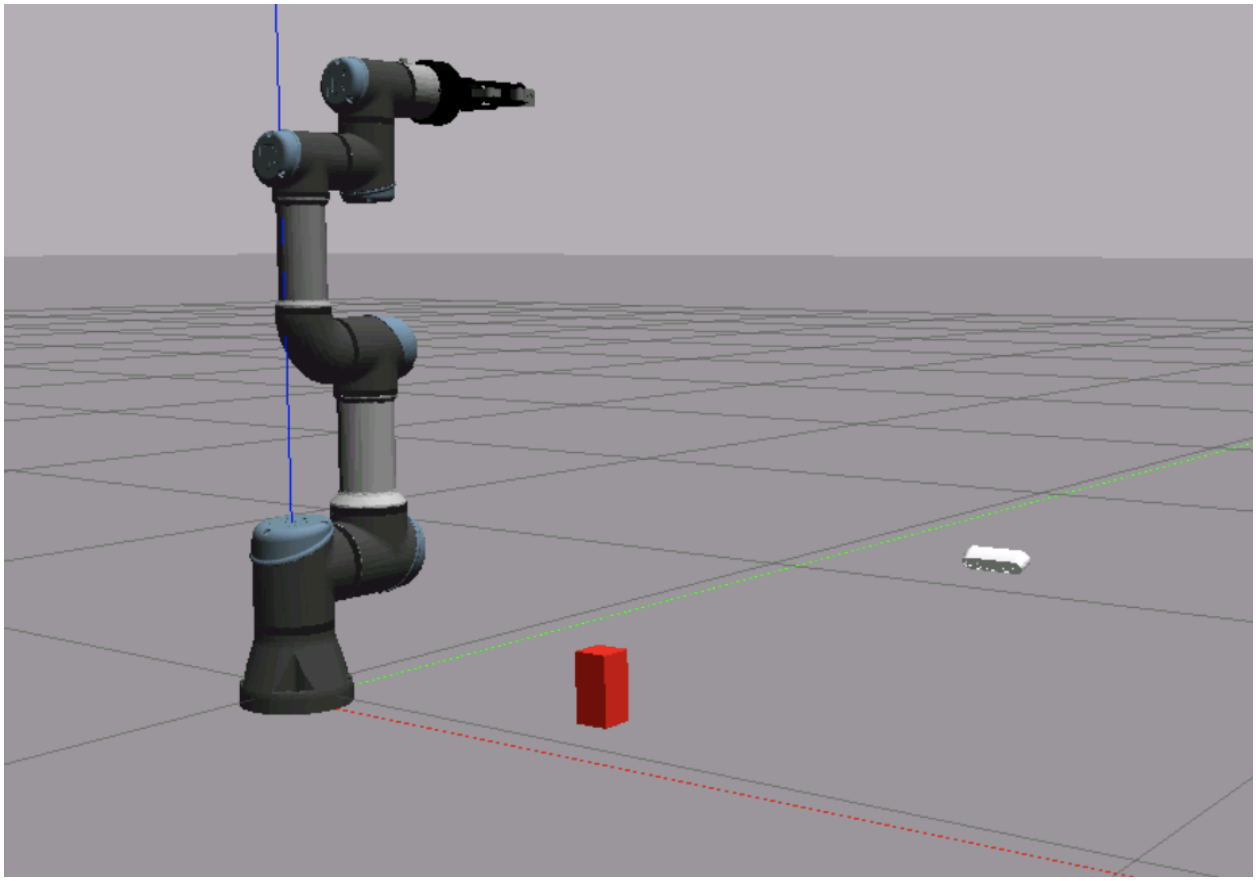
Now, execute the following command to spawn the object into the simulation.

Execute in Shell #2

In [ ]:

```
ros2 run gazebo_ros spawn_entity.py -file ~/ros2_ws/src/grasp_box.urdf -x 0.34 -y 0.13 -z 0.1 -entity grasp_box
```

You will see something like this:



Now, in your **moveit2_scripts** package, create a new file named **approach_retreat.cpp**.
The program generated should perform the following tasks:

- Go to the **Home** position.
- Go to a pose near the object. For this, you can use the following pose:

In [ ]:

```
target_pose.orientation.x = -1.0;
target_pose.orientation.y = 0.00;
target_pose.orientation.z = 0.00;
target_pose.orientation.w = 0.00;
target_pose.position.x = 0.343;
target_pose.position.y = 0.132;
target_pose.position.z = 0.264;
```

- Open the gripper.
- Approach the object. Use a `delta` variable in order to move the end-effector down 4 centimeters.

In [ ]:

```
target_pose.position.z = target_pose.position.z - delta;
```

- Close the gripper for picking the object.
- Retreat. Use a `delta` variable in order to move the end-effector up 4 centimeters.

In [ ]:

```
target_pose.position.z = target_pose.position.z + delta;
```

- Open the gripper to release the object.

b) Create a launch file named **approach_retreat.launch.py** that start your program.
c) Run the following commands to complete the Approach & Retreat sequence:

Execute in Shell 1

In [ ]:

```
ros2 launch my_moveit_config move_group.launch.py
```

Execute in Shell 2

In [ ]:

```
ros2 launch my_moveit_config moveit_rviz.launch.py
```

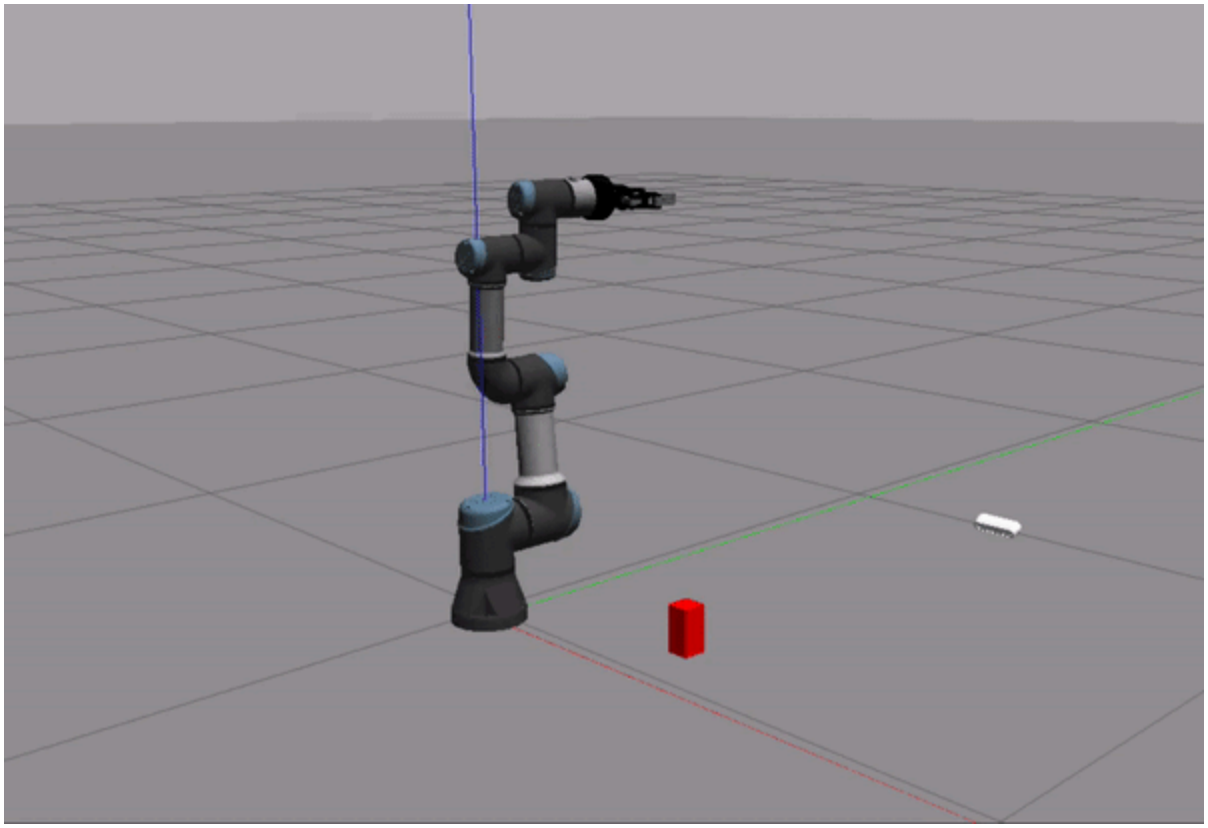
Execute in Shell 3

In [ ]:

```
ros2 launch moveit2_scripts approach_retreat.launch.py
```

- Expected Result for Exercise 3.6 -

IMPORTANT NOTE: The approach & retreat motions might look different for you, but they won't look like in the GIFs shown in the introducion of this section. Keep reading after this exercise to understand why.



- End of Expected Result -

- End of Exercise 3.6 -

- Solution for Exercise 3.6 -

approach_retreat.cpp

In [ ]:

```
#include <moveit/move_group_interface/move_group_interface.h>
#include <moveit/planning_scene_interface/planning_scene_interface.h>

#include <moveit_msgs/msg/display_robot_state.hpp>
#include <moveit_msgs/msg/display_trajectory.hpp>

static const rclcpp::Logger LOGGER = rclcpp::get_logger("move_group_demo");

int main(int argc, char **argv) {
  rclcpp::init(argc, argv);
  rclcpp::NodeOptions node_options;
  node_options.automatically_declare_parameters_from_overrides(true);
  auto move_group_node =
      rclcpp::Node::make_shared("move_group_interface_tutorial", node_options);

  rclcpp::executors::SingleThreadedExecutor executor;
  executor.add_node(move_group_node);
  std::thread([&executor]() { executor.spin(); }).detach();
```

```
static const std::string PLANNING_GROUP_ARM = "ur_manipulator";
static const std::string PLANNING_GROUP_GRIPPER = "gripper";

moveit::planning_interface::MoveGroupInterface move_group_arm(
    move_group_node, PLANNING_GROUP_ARM);
moveit::planning_interface::MoveGroupInterface move_group_gripper(
    move_group_node, PLANNING_GROUP_GRIPPER);

const moveit::core::JointModelGroup *joint_model_group_arm =
    move_group_arm.getCurrentState()->getJointModelGroup(PLANNING_GROUP_ARM);
const moveit::core::JointModelGroup *joint_model_group_gripper =
    move_group_gripper.getCurrentState()->getJointModelGroup(
        PLANNING_GROUP_GRIPPER);

// Get Current State
moveit::core::RobotStatePtr current_state_arm =
    move_group_arm.getCurrentState(10);
moveit::core::RobotStatePtr current_state_gripper =
    move_group_gripper.getCurrentState(10);

std::vector<double> joint_group_positions_arm;
std::vector<double> joint_group_positions_gripper;
current_state_arm->copyJointGroupPositions(joint_model_group_arm,
                                           joint_group_positions_arm);
current_state_gripper->copyJointGroupPositions(joint_model_group_gripper,
                                               joint_group_positions_gripper);

// Go Home
move_group_arm.setStartStateToCurrentState();
move_group_gripper.setStartStateToCurrentState();
RCLCPP_INFO(LOGGER, "Going Home");

// joint_group_positions_arm[0] = 0.00;  // Shoulder Pan
joint_group_positions_arm[1] = -2.50; // Shoulder Lift
joint_group_positions_arm[2] = 1.50;  // Elbow
joint_group_positions_arm[3] = -1.50; // Wrist 1
joint_group_positions_arm[4] = -1.55; // Wrist 2
// joint_group_positions_arm[5] = 0.00;  // Wrist 3

move_group_arm.setJointValueTarget(joint_group_positions_arm);

moveit::planning_interface::MoveGroupInterface::Plan my_plan_arm;
bool success_arm = (move_group_arm.plan(my_plan_arm) ==
                    moveit::core::MoveItErrorCode::SUCCESS);

move_group_arm.execute(my_plan_arm);
```

```
// Pregrasp

RCLCPP_INFO(LOGGER, "Pregrasp Position");
current_state_arm = move_group_arm.getCurrentState(10);
current_state_gripper = move_group_gripper.getCurrentState(10);
current_state_arm->copyJointGroupPositions(joint_model_group_arm,
                                           joint_group_positions_arm);
current_state_gripper->copyJointGroupPositions(joint_model_group_gripper,
                                               joint_group_positions_gripper);

geometry_msgs::msg::Pose target_pose1;
target_pose1.orientation.x = -1.0;
target_pose1.orientation.y = 0.00;
target_pose1.orientation.z = 0.00;
target_pose1.orientation.w = 0.00;
target_pose1.position.x = 0.343;
target_pose1.position.y = 0.132;
target_pose1.position.z = 0.264;
move_group_arm.setPoseTarget(target_pose1);

success_arm = (move_group_arm.plan(my_plan_arm) ==
               moveit::core::MoveItErrorCode::SUCCESS);

// Execute
move_group_arm.execute(my_plan_arm);

// Open Gripper

RCLCPP_INFO(LOGGER, "Open Gripper!");

move_group_gripper.setNamedTarget("open");

moveit::planning_interface::MoveGroupInterface::Plan my_plan_gripper;
bool success_gripper = (move_group_gripper.plan(my_plan_gripper) ==
                        moveit::core::MoveItErrorCode::SUCCESS);

// Execute
move_group_gripper.execute(my_plan_gripper);

// Approach

current_state_arm = move_group_arm.getCurrentState(10);
current_state_gripper = move_group_gripper.getCurrentState(10);
current_state_arm->copyJointGroupPositions(joint_model_group_arm,
                                           joint_group_positions_arm);
```

```
current_state_gripper->copyJointGroupPositions(joint_model_group_gripper,
                                        joint_group_positions_gripper);
RCLCPP_INFO(LOGGER, "Approach to object");

float delta = 0.04;
target_pose1.orientation.x = -1.0;
target_pose1.orientation.y = 0.00;
target_pose1.orientation.z = 0.00;
target_pose1.orientation.w = 0.00;
target_pose1.position.x = 0.343;
target_pose1.position.y = 0.132;
target_pose1.position.z = target_pose1.position.z - delta;
move_group_arm.setPoseTarget(target_pose1);

success_arm = (move_group_arm.plan(my_plan_arm) ==
            moveit::core::MoveItErrorCode::SUCCESS);

// Execute
move_group_arm.execute(my_plan_arm);

// Close Gripper

RCLCPP_INFO(LOGGER, "Close Gripper!");

move_group_gripper.setNamedTarget("close");

success_gripper = (move_group_gripper.plan(my_plan_gripper) ==
                moveit::core::MoveItErrorCode::SUCCESS);

// Execute
move_group_gripper.execute(my_plan_gripper);

// Retreat

current_state_arm = move_group_arm.getCurrentState(10);
current_state_gripper = move_group_gripper.getCurrentState(10);
current_state_arm->copyJointGroupPositions(joint_model_group_arm,
                                     joint_group_positions_arm);
current_state_gripper->copyJointGroupPositions(joint_model_group_gripper,
                                        joint_group_positions_gripper);

target_pose1.orientation.x = -1.0;
target_pose1.orientation.y = 0.00;
target_pose1.orientation.z = 0.00;
target_pose1.orientation.w = 0.00;
target_pose1.position.x = 0.343;
```

```
  target_pose1.position.y = 0.132;
  target_pose1.position.z = target_pose1.position.z + delta;
  move_group_arm.setPoseTarget(target_pose1);

  success_arm = (move_group_arm.plan(my_plan_arm) ==
                 moveit::core::MoveItErrorCode::SUCCESS);

  // Execute
  move_group_arm.execute(my_plan_arm);

  // Open Gripper

  RCLCPP_INFO(LOGGER, "Open Gripper!");

  move_group_gripper.setNamedTarget("open");

  success_gripper = (move_group_gripper.plan(my_plan_gripper) ==
                     moveit::core::MoveItErrorCode::SUCCESS);

  // Execute
  move_group_gripper.execute(my_plan_gripper);

  rclcpp::shutdown();
  return 0;
}
```

- End Solution for Exercise 3.6 -

I know what you are asking yourself right now. Why is the arm performing all these complex motions when I just want it to move a little bit? And the answer is because of inverse kinematics (IK). Since you are setting a new end-effector pose, new IK solutions are computed, which result in overly complex motions.

So, the next question you are surely asking yourself is... can this behavior be changed in some way? And the answer is... YES! But for that, you will need to continue with the next unit!

English proofread

In [ ]: